Global Optimizations

KC Sivaramakrishnan

IIT Madras

Monsoon 2026

Types of optimizations

Classification by scope

- **Local**: optimizes within a single basic block.
  Seen in the previous lecture: constant folding and dead code elimination.
- **Global intra-procedural**: optimizes across basic blocks within one function, using its control-flow graph.
- **Global inter-procedural**: optimizes across function boundaries, using information about callers and callees.

Classification based on their positioning:

- High level optimizations (use the program structure to optimize).
- Low level optimizations (work on medium/lower level IR)

Optimization classification (contd)

Classification with respect to their dependence on the target machine.

Machine independent

- applicable across broad range of machines
- Examples
  - move evaluation to a less frequently executed place
  - remove redundant (unreachable, useless) code.
- create opportunities.

Machine dependent

- capitalize on machine-specific properties
- improve mapping from IR onto machine
- strength reduction.
- replace sequence of instructions with more powerful one (use “exotic” instructions)

Optimization

Desirable properties of an optimizing compiler

- code at least as good as an assembler programmer
  - Match the speed and code size of expert hand-written assembly.
- stable, robust performance (predictability)
  - Small source changes should not cause large performance swings.
- architectural strengths fully exploited
  - Use registers, vector instructions, and parallel execution units.
- architectural weaknesses fully hidden
  - Reduce stalls by scheduling work around slow operations.
- broad, efficient support for language features
  - Keep features such as function calls and objects inexpensive.
- instantaneous compilation
  - An ideal: spend little compile time while improving generated code.

From local to global optimization

Recall the local basic-block optimizations:

- Constant propagation
- Dead code elimination

Original

```
x = 3
y = z * w
q = x + y
```

Propagate x = 3

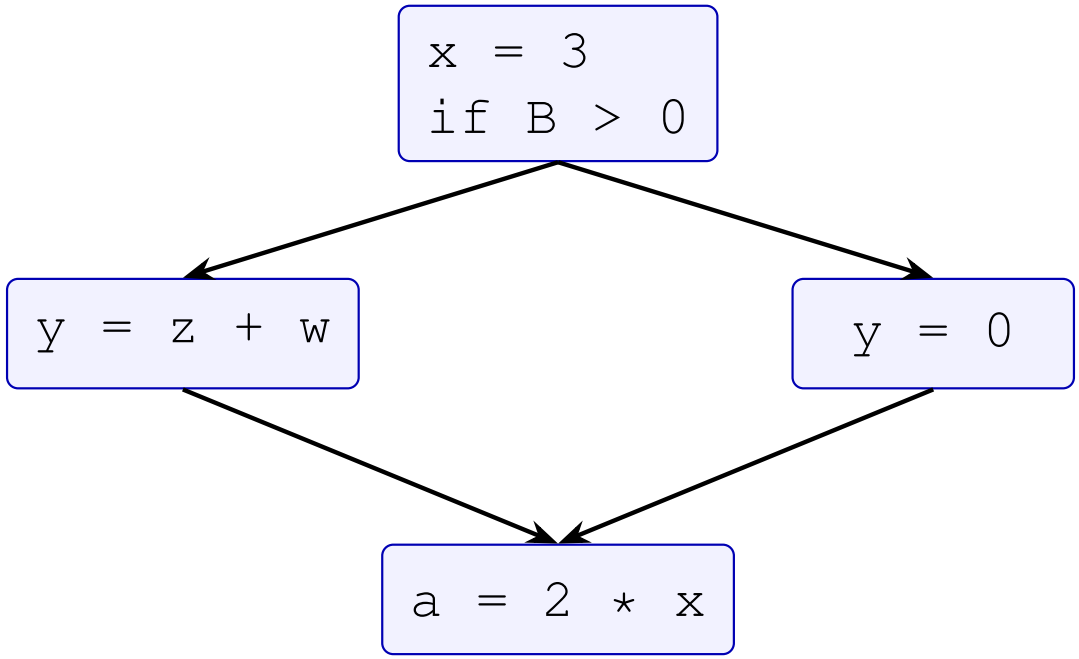
```
x = 3
y = z * w
q = 3 + y
```

Remove dead x

```
y = z * w
q = 3 + y
```

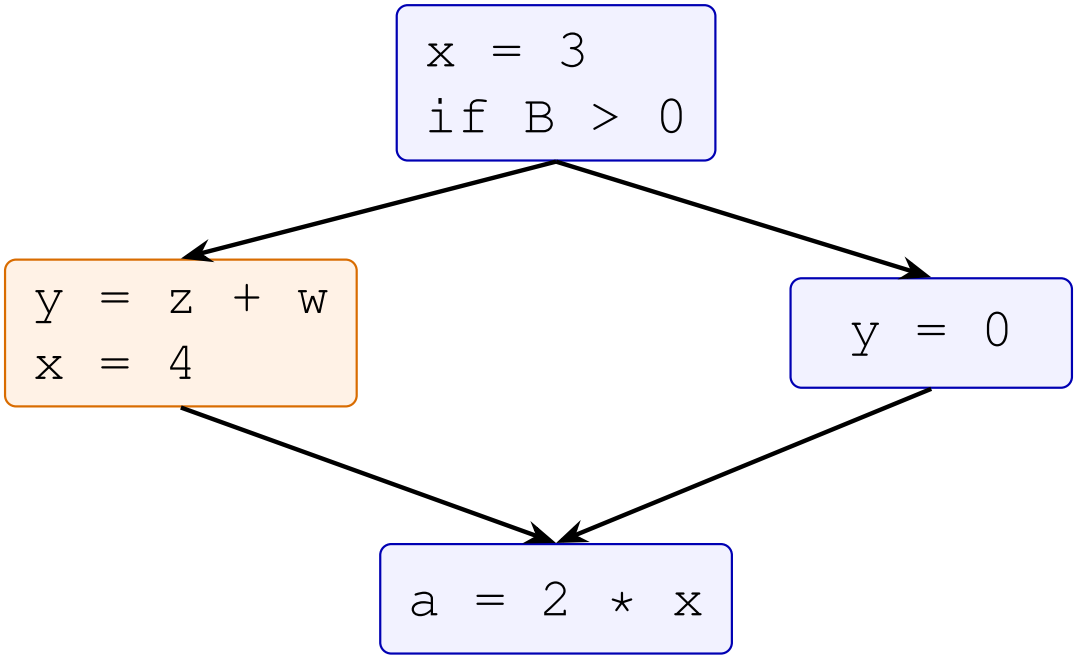
From local to global optimization

Can these optimizations be directly extended to an entire control-flow graph?



From local to global optimization

Can these optimizations be directly extended to an entire control-flow graph? There are situations where it is incorrect to globally propagate constants:



From local to global optimization: Constant Propagation

Correctness Criterion

To replace a use of x by constant k, on every path to the use of x, the last assignment to x is $x = k$.

- This correctness criterion is non-trivial to check.
- 'Every path' includes paths around loops and through branches of conditionals.
- This requires a 'global' analysis, i.e. an analysis of the entire control-flow graph.

Global Analysis

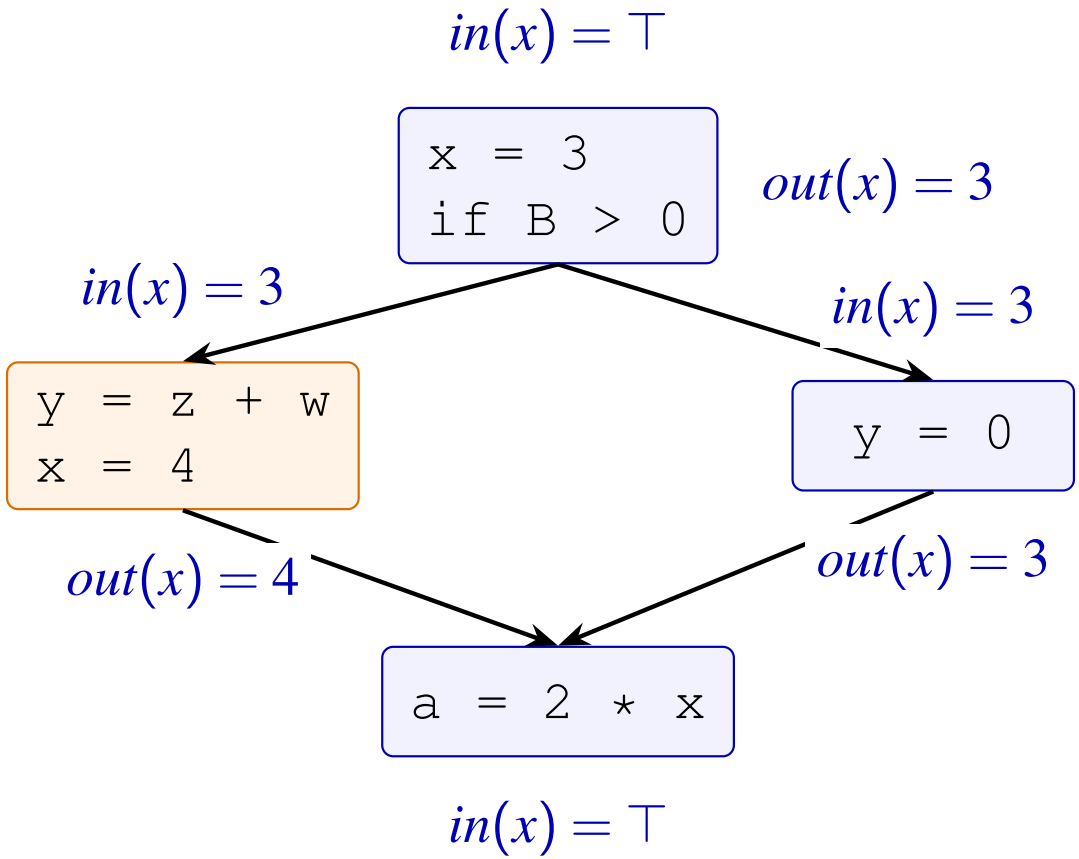
Global optimization tasks share several traits:

- The optimization depends on knowing some property X at a particular point in program execution.
- Proving X at any point requires knowledge of the **entire function**.
- It is OK to be **conservative**. If the optimization requires X to be true, then the analysis can output one of two things:
  - X is **definitely true**
  - **Don't know** if X is true.
- **Global dataflow analysis** is a standard technique for performing global optimizations.

Global Constant Propagation

- To perform global constant propagation at a program point, we need to know whether a variable will always have a constant value at the point.
- We associate one of the following values with variable x at every program point:
  - $\perp$: No reachable path to this point has been discovered yet.
  - c: x has the same value c on every path reaching this point.
  - $\top$: No single constant value is known for x.

Example


$in(x) = \top$
x = 3
if B > 0
$out(x) = 3$
$in(x) = 3$
$in(x) = 3$
y = z + w
x = 4
y = 0
$out(x) = 4$
$out(x) = 3$
a = 2 * x
$in(x) = \top$

Defining Global Constant Propagation Analysis

- We associate two functions $in, out : Var \rightarrow \mathbb{Z} \cup \{\top, \bot\}$ with each basic block.
- Global Constant Propagation Analysis is a forward analysis: in of a basic block is defined in terms of out of predecessors.
- For a basic block b, In_b is defined as follows:

Let $S = \{out_p(x) \mid p \in Pred(b)\}$ be the incoming values for x.

$$in_b(x) = \begin{cases} \bot & S \subseteq \{\bot\} \\ c & S \setminus \{\bot\} = \{c\}, \quad c \in \mathbb{Z} \\ \top & \text{otherwise.} \end{cases}$$

The above definition is also called a meet operation.
We use the following notation for the above definition:
$in_b = \bigvee_{p \in Pred(b)} out_p$

Constant propagation within a basic block

- We now analyse **the statements within basic block** b: starting from in_b, compute out_b.
- For simplicity, assume a separate basic block for each statement.

$$out_b(x) = \begin{cases} \bot & in_b(x) = \bot \\ c & b: \texttt{x} = \texttt{c} \text{ where } \texttt{c} \in \mathbb{Z} \\ \top & b: \texttt{x} = \texttt{e} \text{ where } \texttt{e} \text{ is an expression} \\ In_b(x) & b: \texttt{y} = \ldots \end{cases}$$

Transfer function

We can express the definition as a function f_b such that

$$f_b(in_b) = out_b.$$

With one statement per basic block, f_b describes the effect of that statement.

Defining Global Constant Propagation Analysis

$$out_b(x) = \begin{cases} \bot & in_b(x) = \bot \\ c & b: \texttt{x = c}, \quad c \in \mathbb{Z} \\ \text{eval}(e, in_b) & b: \texttt{x = e}, \quad e \text{ is an expression} \\ in_b(x) & b: \texttt{y = } \ldots \end{cases}$$

We can also do constant folding while evaluating expressions.

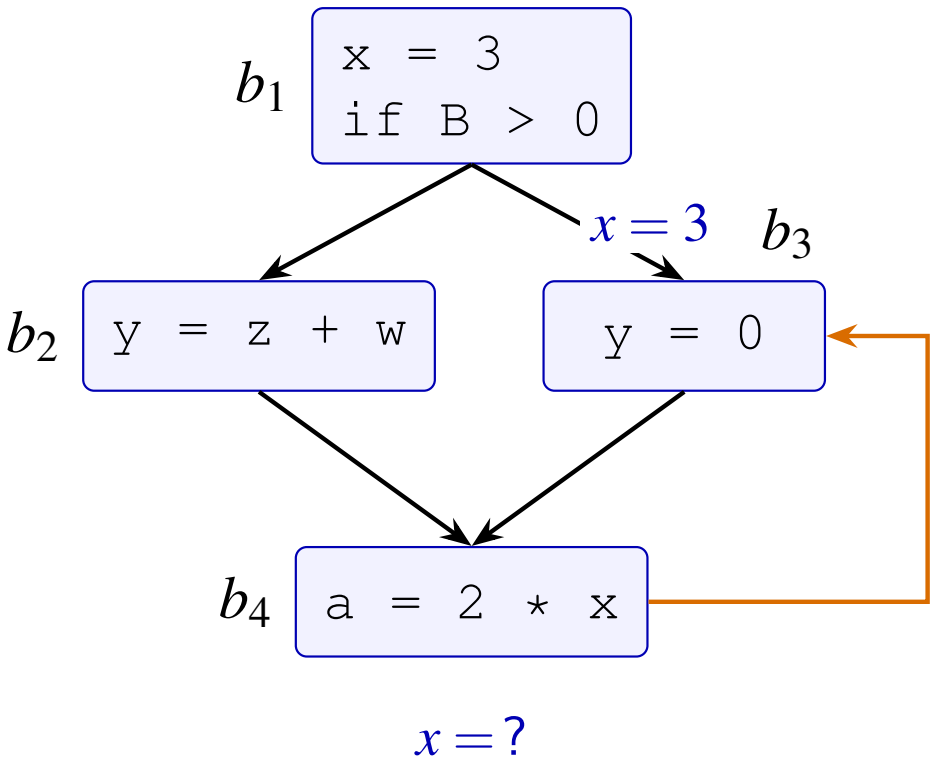
- $\text{eval}(e, f)$ evaluates expression e using the abstract values assigned to variables by f.
- This is **abstract interpretation**: compute with constants, $\bot$, and $\top$ instead of concrete runtime values.
- For example, $3 + 2 = 5$, but $\top + 2 = \top$: an unknown value generally prevents constant folding.

Iterative method for computing In, Out

N: CFG nodes; $Start$: successors of `entry`.

foreach $n \in Start$ **do**
  $in_n \leftarrow \lambda v.\top$; // Incoming values are unknown
end
foreach $n \in N - Start$ **do**
  $in_n \leftarrow \lambda v.\bot$; // No path discovered yet
  $out_n \leftarrow \lambda v.\bot$;
end
repeat // Stop at a fixed point
  foreach n ∈ Nodes **do**
    $in'_n \leftarrow in_n$; // Save values to detect changes
    $out'_n \leftarrow out_n$;
    $in_n \leftarrow \bigvee_{p \in Pred(n)} out_p$; // Merge predecessor information
    $out_n \leftarrow f_n(in_n)$; // Apply the block transfer function
  end
until $\forall n, in'_n = in_n \wedge out'_n = out_n$;

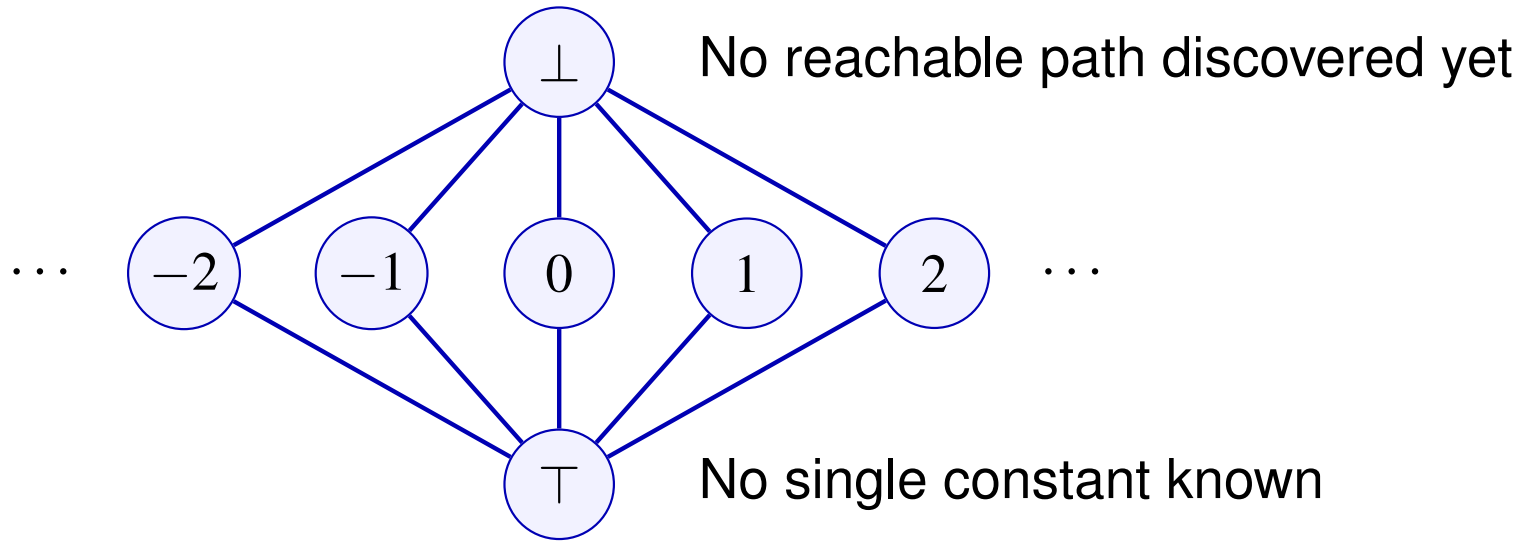
Why do we need $\perp$?



- To compute in_{b_3}, we need out_{b_4}, but for that we need out_{b_3}!
- We will encounter similar problems whenever we have loops.
- Hence, we initialize in and out values with $\perp$, which intuitively means that 'so far as we know, control never reaches this point'

Orderings: the constant-propagation lattice

- $in_b, out_b : Var \to D$, where $D = \mathbb{Z} \cup \{\bot, \top\}$.
- We use the partial order $\top \leq c \leq \bot$ for every $c \in \mathbb{Z}$.
- Distinct constants are **incomparable**: neither $2 \leq 3$ nor $3 \leq 2$.



Higher in the diagram means greater under $\leq$. The symbols $\bot$ and $\top$ follow the convention used in this deck.

Orderings: meet combines predecessor values

- The **meet** is the greatest lower bound (glb): the highest value that lies below every input in the lattice.
- At a control-flow join, compute the meet separately for each variable:

$$in_b(x) = \text{glb}\{out_p(x) \mid p \in Pred(b)\}.$$

Rules

- Ignore $\bot$ inputs; if all are $\bot$, the result is $\bot$.
- If the remaining inputs are the same constant c, return c.
- Different constants, or any $\top$ input, give $\top$.

Examples

$$\text{glb}\{3, \bot\} = 3$$
$$\text{glb}\{3, 3\} = 3$$
$$\text{glb}\{3, 4\} = \top$$
$$\text{glb}\{3, \top\} = \top$$

Constant propagation: meet and transfer

A dataflow analysis has two core ingredients: **combine information across paths** and **transform it through code**.

Meet: combine predecessor information

$$in_b(x) = \text{glb}\{out_p(x) \mid p \in Pred(b)\}.$$

For example, $\text{glb}\{3, \bot\} = 3$, but $\text{glb}\{3, 4\} = \top$.

Transfer function: account for the statement in b

For a reachable block `x = e`, $out_b = f_b(in_b)$ sets x to $\text{eval}(e, in_b)$; other variables keep their input values. An unreached input stays unreached.

Example: `x = y + 2` with $in_b(y) = 3$ gives $out_b(x) = 5$.

This is a **forward** analysis over the constant-propagation lattice. Use unknown values at entry, initialize other nodes to $\bot$, and iterate to a fixed point.

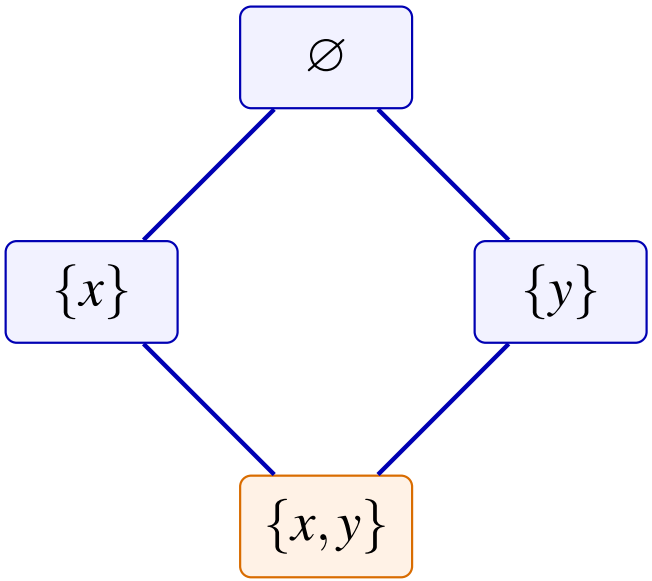
Orderings

- Every data-flow analysis can be represented by defining its meet semi-lattice, with the corresponding meet operation being used in the iterative method.
  - Useful for proving the soundness of the analysis, for comparing precision of different analyses, and for proving termination of the iterative method.
  - Dataflow analysis/Abstract Interpretation covered in detail in advanced courses: CS5030, CS6013.
- Termination argument: We start with the highest value ($\perp$) and we only move down.
  - $\perp$ can change to a constant value, which can change to $\top$.
  - Thus, each $in(x)$ or $out(x)$ can change at most twice at any basic block.
  - Maximum number of iterations = 2 * 2 * Number of variables * Number of basic blocks.

Liveness analysis: the lattice

The abstract values are sets of variables that may be live: $(\mathbb{P}(Var), \supseteq)$.

Example: $Var = \{x, y\}$



meet of $\{x\}$ and $\{y\}$

Reverse inclusion

$$S \preceq T \iff S \supseteq T$$

- Moving down adds variables.
- $\varnothing$ is greatest; Var is least.
- $\{x\}$ and $\{y\}$ are incomparable.

Meet is union

$$\text{glb}(\{x\}, \{y\}) = \{x, y\}.$$

It is the highest set below both inputs in this ordering.

Liveness analysis: backward propagation

- A variable is live on exit if its current value may be used **before redefinition** along at least one successor path. Combine successor information using union:

$$out_b = \bigcup_{s \in succ(b)} in_s.$$

For example, successor live-in sets $\{x\}$ and $\{y\}$ give $out_b = \{x, y\}$.

Transfer function

$$in_b = f_b(out_b), \qquad f_b(S) = use_b \cup (S \setminus def_b).$$

Remove variables defined in the block; add those used before definition.

- use_b: variables used before any definition of them in b.
- def_b: variables defined in b.

Types of program analysis

Classification of analysis (based on their view)

```
if (cond) {
    a = ...
    b = ...
} else {
    a = ...
    c = ...
}
// Which of the variables may be assigned? -- {a,b,c}
// Which of the variables must be assigned? -- {a}
```

- May analysis – the analysis holds on **at least one** data flow path.
- Must analysis – the analysis must hold **on all** data flow paths.
  - What can we say about constant propagation analysis? May or Must?
  - What can we say about liveness analysis? May or Must?

May or must? Answers

Constant propagation is a must analysis

- Replace x by c only when $x = c$ holds **on all** reachable paths to that program point.
- At a join, incoming values 3 and 3 give 3; incoming values 3 and 4 give $\top$ (no single constant).

Liveness is a may analysis

- Keep x live if its current value may be used before redefinition along **at least one** successor path.
- At a branch, successor live-in sets $\{x\}$ and $\varnothing$ give $out_b = \{x\} \cup \varnothing = \{x\}$.

Classification of analysis (contd)

- **Flow-insensitive**: ignores statement order; computes one result for the whole fragment.
- **Flow-sensitive**: tracks separate facts at each program point.

```
if (cond) {
  a = 2;
  b = a;
  c = 3;
  print(a, b, c); // P1
} else {
  a = 3;
  b = a;
  c = 3;
  print(a, b, c); // P2
}
```

At each `print`, which variables are known to be constant under each analysis?

Flow-sensitive
P1: $a = 2,\ b = 2,\ c = 3$.
P2: $a = 3,\ b = 3,\ c = 3$.

Flow-insensitive
Only $c = 3$; a and b may be 2 or 3.

Why use flow-insensitive analysis?

- **Lower cost**: one summary can use less time and memory than separate facts at every program point.
- **Still useful**: in the previous example, it proves $c = 3$, allowing us to replace c by 3 at both `print`s.
  - It misses the constants for a and b, so some optimization opportunities are lost.
- This **cost–precision trade-off** is useful in practice (cf. alias analysis, introduced later).

Classification of analysis (contd)

- **Context-sensitive**: keeps separate facts for different calling contexts, such as different call sites.
- **Context-insensitive**: merges information from all calls to a function into one shared analysis result.

Example: propagate constants across function calls. Here, each call site is a context. What constants can we infer at `print`?

```
a = foo(2); // F1
b = foo(3); // F2
c = bar(2); // B1
d = bar(2); // B2
print(a, b, c, d);

int foo(int x) { return x; }
int bar(int x) { return x * x; }
```

Context sensitivity: explaining the example

We use constant propagation with argument and return-value facts. The context is the **call site**, not the argument value.

Keep calling contexts separate

- At F1, `foo` receives $x = 2$ and returns 2, so $a = 2$. At F2, it receives $x = 3$ and returns 3, so $b = 3$.
- B1 and B2 are distinct contexts, but both pass 2 to `bar`: $x * x = 4$, so $c = d = 4$.

Merge calls to each function

- For `foo`, merge 2 and 3 to $\top$. Its return value is therefore $\top$: neither a nor b is known to be constant.
- For `bar`, both inputs are 2, so the merged input stays 2 and its return value is 4: $c = d = 4$.

Why this example? Merging different inputs can lose constants; merging identical inputs need not.

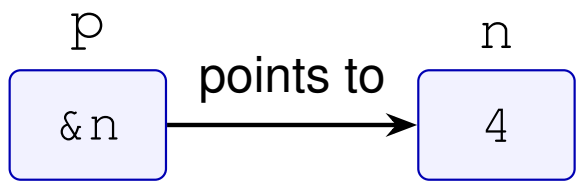
Alias Analysis

Alias analysis: problem of identifying storage locations that can be accessed by more than one way.

Are variable `a` and `b` aliases? $\Rightarrow$ `a` and `b` refer to the same location? Modifying the contents of `a`, modifies the contents of `b`.

Necessary for performing many optimizations such as constant/copy propagation, common sub-expression elimination, dead code elimination, etc.

```
foo(){
  int *p;
  int n;
  p = &n;
  n = 4;
  print ("%d", *p);
}
```



Alias analysis (contd)

```
int *q;
void f(int a, int *p);

int foo(void) {
  int a = 0, k;
  k = a + 5;
  f(a, &k);
  *q = 13;
  k = a + 5;
  return k;
}
```

Is the final **assignment** redundant?
Is the **expression** $a+5$ redundant?
Suppose `f` is implemented as:

```
void f(int a, int *p) {
  q = p;
}
```

Now `q == &k`, so `*q = 13` changes k to 13.

$a+5$ is still 5, but the **assignment is needed** to restore $k=5$.

Loop unrolling

(Example) Matrix-matrix multiply

```
do i = 1, n, 1
  do j = 1, n, 1
    c(i,j) = 0
    do k = 1, n, 1
      c(i,j) = c(i,j) + a(i,k) * b(k,j)
```

- All the array elements are floating point values.
- $2n^3$ flops, n^3 loop increments and branches
- each iteration does 3 loads and 2 flops

Example: loop unrolling

Matrix-matrix multiply: unroll the inner loop by four.

```
do i = 1, n, 1
  do j = 1, n, 1
    c(i,j) = 0
    do k = 1, n, 4
      c(i,j) = c(i,j) + a(i,k)   * b(k,j)
      c(i,j) = c(i,j) + a(i,k+1) * b(k+1,j)
      c(i,j) = c(i,j) + a(i,k+2) * b(k+2,j)
      c(i,j) = c(i,j) + a(i,k+3) * b(k+3,j)
```

- Four original iterations per loop body: the same $2n^3$ flops, with $n^3/4$ inner-loop increments and tests.
- Keep `c(i,j)` in a register; load later operands early to overlap memory access with computation.
- Cache behaviour depends on the **memory layout**, examined next.

Assume n is a multiple of 4; otherwise handle a remainder loop.

Unrolling and cache locality

Assume **row-major storage**, one word per element, and a **four-word cache line**.

One aligned cache line of a

$a(i,k)$	$a(i,k+1)$	$a(i,k+2)$	$a(i,k+3)$

- **Spatial locality in** a: a miss fetching $a(i,k)$ brings the whole line into cache. The next three loads can hit that line.
- **Strided access in** b: $b(k,j)$ and $b(k+1,j)$ are n words apart. These four accesses may need four different cache lines.
- The original loop also has this locality in a. **Unrolling alone does not fix the stride in** b. Next, we reuse each loaded b value across four output rows.

Example: loop unrolling

Same colour = same `b` value, reused across four output rows.

```
do j = 1, n, 1
  do i = 1, n, 4
    c(i,j) = c(i+1,j) = c(i+2,j) = c(i+3,j) = 0
    do k = 1, n, 4
      c(i,j) = c(i,j) + a(i,k) * b(k,j)
        + a(i,k+1)*b(k+1,j)
        + a(i,k+2)*b(k+2,j) + a(i,k+3)*b(k+3,j)
      c(i+1,j) = c(i+1,j) + a(i+1,k) * b(k,j)
        + a(i+1,k+1)*b(k+1,j)
        + a(i+1,k+2)*b(k+2,j) + a(i+1,k+3)*b(k+3,j)
      c(i+2,j) = c(i+2,j) + a(i+2,k) * b(k,j)
        + a(i+2,k+1)*b(k+1,j)
        + a(i+2,k+2)*b(k+2,j) + a(i+2,k+3)*b(k+3,j)
      c(i+3,j) = c(i+3,j) + a(i+3,k) * b(k,j)
        + a(i+3,k+1)*b(k+1,j)
        + a(i+3,k+2)*b(k+2,j) + a(i+3,k+3)*b(k+3,j)
```

Assume n is a multiple of 4; otherwise handle the remaining iterations.

Example: loop unrolling

What happened?

- interchanged `i` and `j` loops
- unrolled `i` loop
- fused inner loops
- $2n^3$ flops, $\frac{n^3}{16}$ loop increments and branches
- first assignment does 9 loads and 8 flops
- 2nd through 4th do 5 loads and 8 flops
- memory traffic is better
  - `c(i+...,j)` is shared across 4 iterations w.r.t the original program
  - `a(i+...,k+...)` references are from cache
  - `b(k+...,j)` is reused (register)

Loop-invariant code motion (LICM)

Loop-invariant computation: produces the same value on every iteration of a loop.

LICM: hoist such a computation before the loop, when doing so preserves program behaviour.

Compute the value once per entry to the loop, then reuse it across iterations.

Example: loop-invariant code motion

```
for i = 1 .. 100
  for j = 1 .. 100
    for k = 1 .. 100
      A[i,j,k] = i * j * k
```

- 3 million index operations
- 2 million multiplications

Example: LICM (cont.)

Hoisting out of the inner loop:

```
for i = 1 .. 100
  for j = 1 .. 100
    t1 = &A[i][j]
    t2 = i * j
    for k = 1 .. 100
      t1[k] = t2 * k
```

And the second loop:

```
for i = 1 .. 100
  t3 = &A[i]
  for j = 1 .. 100
    t1 = &t3[j]
    t2 = i * j
    for k = 1 .. 100
      t1[k] = t2 * k
```

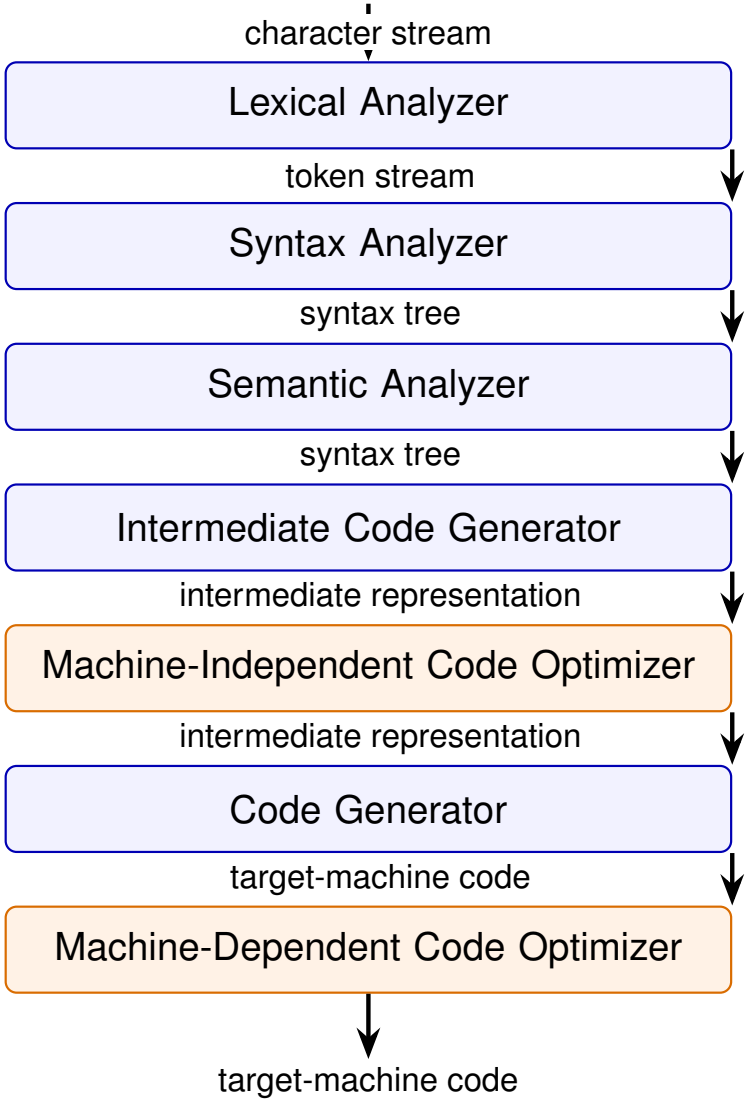
Optimization - overview

Compilers are engineered objects

- minimize running time of compiled code
- minimize compile time
- use reasonable compile-time space
- find a reasonable trade-off

Thus, results are sometimes unexpected

Back to first lecture



Front end responsibilities:

- Recognize syntactically legal code; report errors.
- Recognize semantically legal code; report errors.
- Produce IR.

Back end responsibilities:

- Optimizations, code generation.

Our target

- five out of seven phases.
- glance over optimizations.